

SEN381

Research for Implementati on & Integration Decisions

ASSIGNMENT 2

[Chanice Grant \(601597\)](#)

[Milenke Field \(578122\)](#)

[Megan Heather Wathen \(601611\)](#)

Table of Contents

1. DESIGN QUALITY AND DESIGN PATTERNS

1.1 Introduction

**Research Question → Credible Evidence → Alternatives → Comparison → Recommendation → CivicConnect
Decision → PED/ADR/RTM → Application Evidence**

1.1.1 Purpose of the Research & Relevance to CivicConnect

The research involved with Task 1 is to solve a CivicConnect problem. Firstly, by doing research, then finding and comparing credible alternatives, then considering the advantages and disadvantages of each alternative recommendation that can influence Milestone 2 (M2). Task 1 allows the software engineering team to consider the actual CivicConnect requirements, architecture, constraints, risks, team capabilities, and other project evidence. It is important to note that the recommendations provided by Assignment 2 are not accepted as the final decisions for Milestone 2 (M2). If the software engineering team has defensive project-specific evidence available later to counter the recommendations provided in Assignment 2, then the new observations may be taken as the final decisions for M2.

1.2 Design Quality Foundations

1.2.1 Coupling

Coupling refers to the bond shared between two different modules in terms of how dependent they are on one another. There is said to be coupling present when changing one module will force the developer to change another module (Fowler, 2001). There are two different types of coupling that can exist on a spectrum but are commonly identified as tight and loose. High coupling is present when two classes or modules are heavily reliant on one another, thus changing one thing in class 1 will almost always require you to change something in class 2. Alternatively, loose coupling indicates that the two classes or modules are more independent from one another, thus changing module 1 will not force the developer as much to change module 2 as tight coupling would (Hak, 2026). Both can bring benefits if they are controlled, but loose coupling is preferred for easier testing, more maintainable, more future extensible ability/flexibility, less fragile, and more easily comprehensible code overall (Ergun, 2023). Loose coupling also allows for developers to incorporate good design principles as loose coupling allows for code to be written in compliance with SOLID principles (Ergun, 2023). It is important to note that coupling is not avoidable, only manageable (Ergun, 2023).

An example below of tight coupling shows that changes in the internal structure of one class can affect other classes. In the example below, we want code that logs a message, using classes MessageProcessor and LogManager.

LogManager Class:

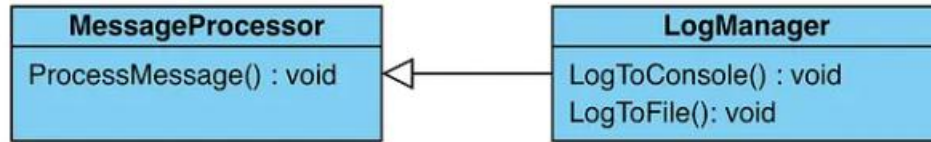


Figure 1: UML Class Diagram to visualize tight coupling examples below (Ergun, 2023).

```
public class LogManager
{
    public void LogToConsole(string message)
    {
        Console.WriteLine($"Logging to console: {message}");
    }

    public void LogToFile(string message)
    {
        Console.WriteLine($"Logging to file: {message}");
    }
}
```

Figure 2: C# code for LogManager Class (Ergun, 2023).

MessageProcessor Class:

```
public class MessageProcessorForTightCoupling
{
    private LogManager logManager;

    public MessageProcessorForTightCoupling()
    {
        this.logManager = new LogManager();
    }

    public void ProcessMessage(string message)
    {
        Console.WriteLine($"Processing message: {message}");
        this.logManager.LogToFile($"Message processed: {message}");
    }
}
```

Figure 3: C# code for MessageProcessor Class for Tight Coupling (Ergun, 2023).

Program.cs (Ergun, 2023):

```
MessageProcessorForTightCoupling messageProcessor = new
MessageProcessorForTightCoupling();
messageProcessor.ProcessMessage("Sample message");
```

This tightly coupled example shows that MessageProcessor depends heavily on LogManager since MessageProcessor cannot perform its tasks without LogManager. If any updates are created in LogManager then immediate updates and maintenance are required in MessageProcessor.

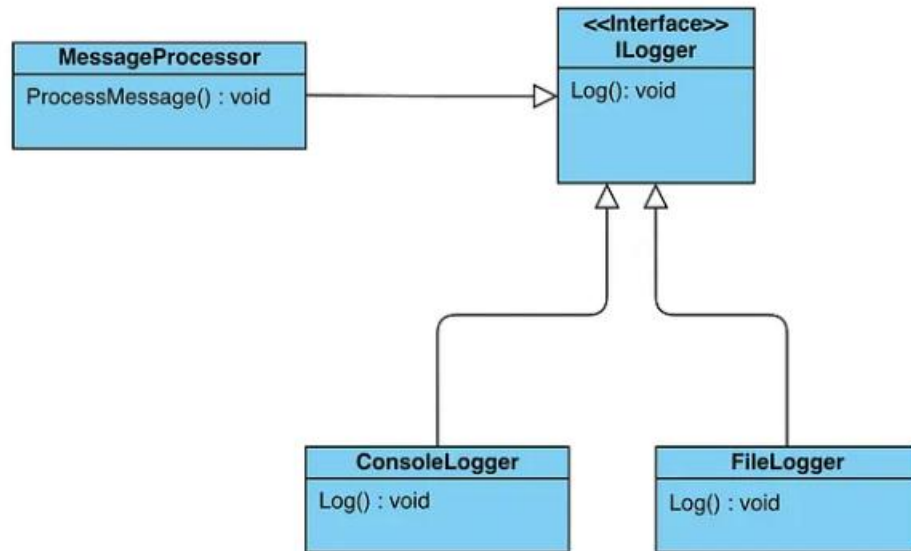


Figure 4: UML Class Diagram to visualize loose coupling below (Ergun, 2023).

ILogger Interface:

```
public interface ILogger
{
    void Log(string message);
}
```

Figure 5: C# code for ILogger Interface (Ergun, 2023).

MessageProcessor Class:

```
public class MessageProcessor
{
    private ILogger logger;

    public MessageProcessor(ILogger logger)
    {
        this.logger = logger;
    }

    public void ProcessMessage(string message)
    {
        Console.WriteLine($"Processing message: {message}");
        this.logger.Log($"Message processed: {message}");
    }
}
```

Figure 6: C# code for MessageProcessor Class for Loose Coupling (Ergun, 2023).

ConsoleLogger Class:

```
public class ConsoleLogger : ILogger
{
    public void Log(string message)
    {
        Console.WriteLine($"Logging to console: {message}");
    }
}
```

Figure 7: C# code for ConsoleLogger Class (Ergun, 2023).

FileLogger Class:

```
public class FileLogger : ILogger
{
    public void Log(string message)
    {
        Console.WriteLine($"Logging to file: {message}");
    }
}
```

Figure 8: C# code for FileLogger Class (Ergun, 2023).

To get rid of the negative effects of tight coupling, more loose coupling can be adopted. The loose coupling example above alters the structure from the tightly coupled example given earlier. The loosely coupled example relates classes and modules through abstract classes or an interface, instead of having one class directly instantiate another class like in the tight coupling example (Adewole, 2018).

1.2.2 Cohesion

Cohesion describes how closely the responsibilities within a class or module or function relate to one another (Izadkhah & Hooshyar, 2017). Cohesion is also measured on a spectrum in terms of a stronger and weaker bond of responsibilities involved. A class with high cohesion has its functions focused on one closely related purpose/responsibility, whereas low cohesion has a class where its functions have to meet too many unrelated responsibilities at once. Chung, Tai, & Chong (2021) discovered that a class shows a higher degree of cohesion when has at least two of the following relationships: a) method-method, b) method-attribute, and c) attribute-attribute. High cohesion is preferred as a good engineering practice to make code more comprehensible, maintainability, testable, and reusable ((Izadkhah & Hooshyar, 2017); (Chen, Tai & Chong, 2021)).). Also, an overall combination of low coupling along with high cohesion is known to be considered a well-designed system of quality (Eder, Kappel & Schrefl, 1994).

An example below of low cohesion shows that tries to perform too many tasks at once, assigning itself an unnecessary amount of responsibilities.

```

using System;

// Bad example - Low cohesion
class UtilityClass {
    public void SendEmail(string message) {
        Console.WriteLine("Sending email: " + message);
    }

    public void PrintDocument(string content) {
        Console.WriteLine("Printing: " + content);
    }

    public double CalculateInterest(double principal, double rate) {
        return principal * rate / 100;
    }

    public void SaveToFile(string data) {
        Console.WriteLine("Saving to file: " + data);
    }
}

class Program {
    public static void Main() {
        UtilityClass util = new UtilityClass();

        // This class does too many unrelated things
        util.SendEmail("Hello World");
        util.PrintDocument("Invoice");
        Console.WriteLine("Interest: " + util.CalculateInterest(1000, 5));
        util.SaveToFile("User data");
    }
}

```

Figure 9: C# code for low cohesion example 1 (TutorialsPoint, 2026).

An example below of high cohesion shows that the same functionality is split into highly cohesive classes, properly sharing the responsibilities of the different tasks.

```

using System;

// High cohesion - Each class has a single responsibility
class EmailService {
    public void SendEmail(string message) {
        Console.WriteLine("Sending email: " + message);
    }
}

class FinanceCalculator {
    public double CalculateInterest(double principal, double rate) {
        return principal * rate / 100;
    }
}

class FileManager {
    public void SaveToFile(string data) {
        Console.WriteLine("Saving to file: " + data);
    }
}

class Program {
    public static void Main() {
        EmailService email = new EmailService();
        FinanceCalculator finance = new FinanceCalculator();
        FileManager fileManager = new FileManager();

        email.SendEmail("Hello World");
        Console.WriteLine("Interest: " + finance.CalculateInterest(1000, 5));
        fileManager.SaveToFile("User data");
    }
}

```

Figure 10: C# code for high cohesion example 1 (TutorialsPoint, 2026).

Another separate example below of high cohesion shows functionality is split into cohesive classes, properly sharing the responsibilities of the different tasks. More specifically, all of the methods and constants used within this class are associated with a math operation.

```

using System;

class MathOperations {
    public static void Main() {
        // All Math class members serve mathematical purposes
        double number = -25.7;
        double radius = 5.0;

        Console.WriteLine("Absolute value: " + Math.Abs(number));
        Console.WriteLine("PI constant: " + Math.PI);
        Console.WriteLine("Power calculation: " + Math.Pow(2, 3));
        Console.WriteLine("Circle area: " + (Math.PI * Math.Pow(radius,
2)));
    }
}

```

Figure 11: C# code for high cohesion example 2 (TutorialsPoint, 2026).

1.2.3 Relevant SOLID Principles

Several authors state that coupling and cohesion are essential to the efficient design of high quality object oriented (OOP) software ((Tiwari & Rathore, 2018); (Fowler, 2001)). It is also observed that improper coupling and cohesion can lead to unnecessary complexity developing which can in turn cause issues with testing, increase residual errors, and increase maintenance effort (Heyliger, 2002). The SOLID principle is also present as criteria for making efficient design of high-quality object oriented (OOP) software.

Single Responsibility (S)	A class should have only one reason to change, meaning it should do only one job.
Open/Closed Principle (O)	Software parts should be open for extension but closed for modification.
Liskov Substitution Principle (L)	Child classes must be able to replace parent classes without breaking the program.
Interface Segregation Principle (I)	Do not force a class to use methods or interfaces it does not need.
Dependency Inversion Principle (D)	Depend on abstractions (like interfaces) rather than concrete details or heavy classes.

The SOLID principles relevant to CivicConnect are: Single Responsibility Principle (S), Open/Closed Principle (O), and Dependency Inversion Principle (D). Regarding CivicConnect, ‘S’ would be helpful to consider so that a class responsible for allowing authorised management to review request status must not also be held responsible for allowing requesters to submit requests or allowing authorised staff to resolve or close a service request. Lastly, ‘S’ is relevant because it promotes high cohesion. ‘O’ would be another important principle for CivicConnect when CivicConnect. For example, CivicConnect currently manages service requests via email, telephone calls, WhatsApp messages, spreadsheets, and paper-based records. If a new form of service request input were introduced like files being delivered by Apple Airdrop (uses Bluetooth and peer-to-peer-wifi (AWDL)), then the Apple Airdrop form of service request input would be required not to interfere with the code already developed to handle the original methods of service request inputs (i.e., email, WhatsApp, etc.). ‘D’ would be relevant to CivicConnect to apply to its communication aspects, to include a notification hub interface that connects the different notification classes together so they can interact with one another without depending on each other directly.

1.3 CivicConnect Design Problem 1

1.3.1 Description of the Design Problem

Processing Different Requestor Incidents (Creating Domain Objects):

CivicConnect processes several different request types that fall under but are not limited to: facility faults, damaged equipment, security concerns, IT support, maintenance issues, lost

property, and operational requests. Each request type may have different data fields depending on the request submitted. A request sent on theft for example may require police report, case number, and security footage/photos as evidence inputs to deal with their request. While a request regarding a reported office water leak may require other forms of input evidence such as pipe infrastructure IDs and estimated flow loss values and calculations. Maintenance requests may require physical samples and/or lab results.

1.3.2 Architectural Issue

Tight coupling is observed as a result of creating complex service request domain objects directly within HTTP controller endpoints using conditional statements. Moreover, when updates are performed there will have to be continuous alterations made to the presentation layer code, which will violate the Single Responsibility Principle and the Open/Closed Principle.

1.3.3 Design Approaches

Criteria	Approach 1: Abstract Factory Pattern	Approach 2: Builder Pattern
Intent & Structure	Provides an interface for creating families of related objects w/o specifying concrete classes	Client interacts directly w/ a concrete builder using a fluent API (i.e. method chaining) to create an object step-by-step
Coupling Impact	Low-> Controllers depend exclusively on abstract factory contracts & abstract interfaces	Low-Med-> Client code is completely decoupled from constructor parameter signatures, but remains directly dependent on the public API methods of the specific Builder class
Cohesion Impact	High-> Enforces consistent object families within category-specific concrete factory classes	High-> target class (i.e. TheftIssueReport) class stays clean & focused only on its business logic while all the boilerplate logic (field assignment, object validation, data parsing) is isolated in the Builder class
Maintainability & Extensibility	Adding an entire new issue category family (i.e. theft instance service requests) requires adding a new concrete factory class	High flexibility-> allows new optional attributes to be created by adding new optional methods to the builder, leaving the existing code untouched and avoiding parameter explosion
Testability	High-> concrete factories can be replaced with mock	High-> builders can be pre-configured in unit tests with valid defaults for flexible test

	factories producing test double families	setup; unit tests will only overwrite the specific fields being tested
Complexity	High	Medium
Misuse & Over-Engineering Risk	Complexity overhead and interface rigidity can arise if new requester types are added frequently to the factory interface	Client code assumes responsibility for controlling the assembly sequence, risking incomplete/invalid object states if steps are missed

1.3.4 Comparison of Problem 1 Approaches

	Approach 1: Abstract Factory Pattern	Approach 2: Builder Pattern
Benefits	-ensures structural consistency across object creation w/o binding controllers to concrete class implementations	-provides clean, readable instantiation w/o constructor argument/parameter explosion -encapsulates dynamic variation b/w different incident families
Limits	-can become less readable if requesters have to add very large objects (request types) or if they are deeply nested	-can have a tendency to become difficult to read since it involves creating separate classes for the builder and the object being built -only creates a single object at a time -cannot prevent client from accidentally combining elements of different objects
Appropriate Circumstance for Use	Use when your code needs to work w/ various families of related services but you do not want it to depend on the concrete classes of those services	Use when want to avoid creating many subclasses. Use when want to avoid having constructors w/ long list of attributes. Use when want to avoid creating many constructors in a class.

1.3.5 Recommendation for CivicConnect for Problem 1

While the Builder pattern can provide more customised and complex object creation, it introduces more room for error (from client input) and potentially more overhead since objects take longer and are more custom and potentially require more complex code to be created. For a solution that needs to be scalable, the Builder Pattern is not the most sustainable option to take. Thus, the Abstract Factory Pattern is recommended for CivicConnect because it encapsulates full co-dependent object family creation behind a single call, ensuring structural integrity and low controller coupling. It is also a good pattern to use since it follows the open-closed and single responsibility principles. Using the Abstract Factory pattern may prove to be more suitable for CivicConnect as related objects (different types of requests) can potentially be more easily and closely organised to their related parties than if the Builder pattern were used. Moreover, resolving CivicConnect Problem 1 is not just about managing long lists of optional parameters on a single object (done well by Builder Pattern), but more importantly it also has to be able to handle entirely distinct families of related domain objects based on incident category, which the Abstract Factory Pattern does and the Builder Pattern does not.

1.4 CivicConnect Design Problem 2

1.4.1 Description of the Design Problem

Multi-Channel Requester Alerts (Adapting Vendor APIs):

CivicConnect must send automated alerts across multiple external channels (SMS, WhatsApp, email, etc.) whenever a requester's reported issue changes status.

1.4.2 Architectural Issue

Instantiating concrete notification client SDKs directly inside core business logic forces domain services to adapt whenever an external vendor updates their API or authentication scheme. Furthermore, changing notification providers or adding a new delivery channel requires modifying and re-testing core business components, violating the Dependency Inversion Principle. This is a pertinent problem to address before major implementation since it deals with the visibility need required by the CivicConnect stakeholders.

1.4.3 Design Approaches

Criteria	Approach 1: Observer Pattern	Approach 2: Adapter Pattern
Intent & Structure	Establishes a 1:N dependency where a subject publishes alert events to registered channel listeners	Wraps incompatible provider APIs behind a unified internal interface (i.e. <code>INotificationAdapter</code>)
Coupling Impact	Low-> Domain services broadcast events w/o	Low-> Decouples core app services from third-party vendor APIs

	knowing concrete alert listener details	
Cohesion Impact	Med-High-> Event dispatch and alert routing are extracted into separate listener components	High-> Translating external vendor APIs is completely isolated to individual adapter classes
Maintainability & Extensibility	New alert channels can be added dynamically by subscribing new listeners to the event bus	Adding or swapping notification vendors requires adding a new adapter w/o touching core business logic
Testability	High-> event listeners can be isolated or mocked out during domain service tests	High-> mock interfaces can easily be injected into domain services during unit tests
Complexity	Low	Moderate
Misuse & Over-Engineering Risk	Risk of dynamic execution flows & potential memory leaks if un-subscriptions fail	Over-engineering if only one static external API will ever be used across the system lifecycle

1.4.4 Comparison of Problem 2 Approaches

	Approach 1: Observer Pattern	Approach 2: Adapter Pattern
Benefits	-improves flexibility & communication b/w objects -provides loose coupling b/w subject & observers -observers can be easily added/removed at runtime -ensures automatic & consistent updates to all dependents	-promotes easy code reuse -isolates adaptation, thus keeps classes fixed on core logic -supports multiple interfaces -client talks to adapter interface and not external tools
Limits	-limited applicability -possibility of being less scalable -can become less readable if over-engineered	-limited applicability -possibility of increased overhead -will most likely experience maintenance challenges
Appropriate Circumstance for Use	Use for event notification purposes (when one object changes state & needs to dynamically update many dependent objects)	Use when there are two classes w/ incompatible interfaces that need to work together

1.4.5 Recommendation for CivicConnect for Problem 2

The Observer Pattern is recommended for CivicConnect as the Adapter Pattern may prove to be more complicated than what is practically acceptable to the stakeholders. The Observer pattern will also be more suited to deal with the notification requirements of CivicConnect

regarding request status updates. Although, with this being said, is it difficult to compare these design patterns in a clear-cut manner as both solve different aspects of the same problem. It would be wise to adopt the Observer Pattern first (as it resolves the core stakeholder problem), and then see if the Adapter Pattern can also be implemented in a hybrid fashion if it will better meet the notification requirements. More specifically, the Observer pattern can be used to trigger the notification alert, and the Adapter pattern can handle the connection to the external vendor (i.e. the WhatsApp listener will use a WhatsApp-specific adapter to translate and send the message).

2. PERSISTENCE AND DATA INTEGRITY

2.1 Meaningful Business Operation

2.1.1 Selected CivicConnect Operation

For the persistence and data-integrity analysis, a business operation is chosen: When a service request is changed from one status to another and a corresponding entry in the request-history is recorded.

In CivicConnect, authorized staff members are required to update requests along the way in their lifecycle. For instance, when the work was completed for a request, it could be assigned to a staff member and then resolved. This operation requires more than just the modification of the value of current-status field. Additionally, CivicConnect needs to maintain a consistent history of the request to ensure a record of previous actions and status changes is available.

A successful status transition can then be two logically connected persistence actions, like:

1. Update the request's status: Assigned → Resolved
2. Record the corresponding transition in the request history/audit record

These are one meaningful business operation because the state of the request and the business history describing its state should be consistent with each other.

This operation was chosen because it facilitates CivicConnect's business goals of traceability and accountability. When the status changes or historical data appears to be inconsistent, users and management may not have a reliable history of the progress of a service request through its lifecycle.

Transactions are related to this problem since they are supposed to maintain consistency and integrity of the database when operations are performed on related data. (Cui, et al., 2024) elaborate on the significance of transactions in database management systems and illustrate that transaction-processing faults can lead to wrong database states and cause other major issues.

2.1.2 Partial or Incorrect Update Consequences

If only some of the status-transition operation succeeds, it would be a partial update.

Suppose that Request #105 is in the Assigned state. The request is resolved by an authorised staff member and CivicConnect changes the status of the request to Resolved. But an error happens before the corresponding history record can be successfully stored.

The data obtained might be as follows:

Current Request Record is the same as Request History.

Status: Resolved, Latest recorded status: Assigned

The values in each individual database might seem to be correct but when combined, they tell an inconsistent story about the same service request. CivicConnect would signify that the request has been taken care of without saving the historical evidence of the transition, when and how it took place.

But the other kind of failure is also possible. This history might show that the request has been changed to Resolved, but the update to the request's status does fail and leaves the request as Assigned. This might then lead to conflicting representations of the same request based on management reports and staff views and requester information.

This may have a negative impact on CivicConnect's accountability and traceability. It is unclear what is being requested, management information may be incorrect, and the organisation may lose trust in the request history as evidence of the actions that have been carried out.

Thus, it is important for the persistence design to consider how the related updates will either succeed or fail without leaving a partially updated state. The atomicity problem is the one that will be discussed in 2.2, not assumed at this time.

Database level integrity is also relevant since the application's validation doesn't necessarily mean that invalid states can never be persisted. Huang et al. (2023) researched highly used web applications and found examples where missing constraints caused failures on pages, prevented business logic flow, and led to possible data corruption. Their work shows that much benefit can be gained from thinking about database integrity controls in addition to application-level behaviour.

The implications of a partial or erroneous update for CivicConnect are thus not just an error in the database. It might compromise the trustworthiness of the lifecycle of a service request that CivicConnect is supposed to manage. The responsibilities for transaction, validation and integrity that need to be performed to avoid these states will then have to be assessed before a persistence approach is recommended.

2.2 Engineering Choices

2.2.1 Transaction and Atomicity Requirements

The operation chosen for CivicConnect is a set of two related database operations: updating the status of a service request and marking the change in status in the service request's history. The persistence design needs to consider if these actions need to be treated as a single transaction, as they are one logical business operation.

Database Management Systems use transactions to ensure data consistency and integrity. According to (Cui, et al., 2024), transaction processing failures may lead to incorrect states of

databases and other severe issues. They examined 140 transaction bugs in six popular DBMSs, showing that transaction behaviour directly impacts data correctness.

Atomicity is of special significance for CivicConnect. If the status update is successful, and the history entry fails, or vice versa, only a partial operation may be saved. The engineering requirement is thus that the logically related persistence actions must be finished successfully – either together or not at all – leaving the database in or out of a partial state.

This is not yet a speculating on the exact database technology / implementation technique. Rather, it sets the atomic completion as a property that the alternatives listed in Section 2.2.4 shall be assessed to.

2.2.2 Validation Responsibilities

CivicConnect needs to then identify where validation responsibilities should be applied. Validation can be done at multiple levels not just at the user interface or at the database level.

At the application level, CivicConnect can validate business rules before it will try to persist a change. For instance, the application could be able to decide as to whether the user is authorized to resolve the request, and whether the proposed status transition is allowed. User-interface validation can also discover missing or incorrectly formatted data before it is submitted.

But this only works if the values are wrong at the application level; if they are wrong at the database level (due to some other source of error) or if they are wrong at some other level (due to an application defect), then the values will be lost. (Huang, et al., 2023) discovered that the database integrity constraints often are not fully leveraged by database-backed web applications. Their study found missing constraints with consequences such as application behaviour failure and corrupted data.

This result poses an important engineering decision for CivicConnect. Validation could be done at the application logic level, or it could be split between application-level business validation and database level integrity constraints. The alternatives vary in terms of complexity, maintainability and the level of protection of persisted data.

2.2.3 Consistency, Concurrency and Data Integrity

Data integrity needs to be preserved when more than a person authorised to access the same service request at about the similar time.

Let's take for instance, that Staff Member A and Staff Member B may both be fetching Request #105 when it is Assigned. A member of staff then resolves the request. If Staff Member B is still using the "older" version of the request, and submit an update later, the system must decide what to do with that outdated update. Otherwise, if the other user changed, his change might be overwritten, or he might have done something that conflicts with another change.

The primary reason for concurrency control is that simultaneous operations on shared data may yield inconsistent results if not adequately coordinated. When two processes are operating on the same data element, (Chaudhry & Yousaf, 2022) describe a situation that may lead to data-consistency issues and discuss various concurrency-control strategies used to solve these problems.

Concurrency, for CivicConnect, is more than just supporting multiple users at once. The key engineering problem is the ability to have multiple staff members create incompatible or conflicting request data without realizing it. Consequently, any persistence method chosen should offer a way of detecting, preventing or handling conflicting updates safely.

2.2.4 Alternative Implementation Approaches

There are two general implementation strategies that can be considered for the CivicConnect operation selected.

Approach A — Application-centred control: The application/service layer would perform the main business-rule validation and coordinate the status and history updates. This keeps much of the behaviour within the application code and can make business logic easier for developers to locate and modify. However, greater reliance on application logic means that the database itself may provide less protection if invalid or inconsistent operations reach it.

Approach B — Layered application and database control: Business-specific validation would still occur within the application/service layer, while the database would additionally enforce fundamental integrity rules and transactional behaviour. The status update and history insertion could be performed within one transaction, while suitable database constraints protect important stored-data relationships.

It doesn't imply moving all the business logic into the database using the second approach. Rather, there is a separation of responsibility based on the purpose: the application handles CivicConnect-specific behaviour, and database mechanisms also offer added protection for persistent data.

This distinction is validated by the work of Huang et al., who showed that database constraints can help ensure integrity in the face of errors in application code or operational mistakes, and by the work of (Cui, et al., 2024) on transaction research, showing that the integrity of the database state relies on getting transactions to behave correctly.

There is another design decision added to either of these in the case of concurrency. CivicConnect can implement concurrency control by using locking mechanisms or an optimistic approach in which the concurrency control is implemented by detecting conflicts when trying to update. Choices should be based on the consequences and usage of conflicting changes, in the context of the specific needs of the application, not on the assumption that one concurrency mechanism is best for all applications.

2.2.5 Comparison and Trade-offs

Engineering consideration	Application-centred control	Layered application + database control
Business-rule validation	Primarily application layer	Primarily application layer
Database integrity protection	Greater dependence on correct application behaviour	Additional protection through database constraints
Atomic status + history update	Must be coordinated correctly by application persistence logic	Can be protected as a database transaction

Maintainability	Business behaviour concentrated in application code	Responsibilities divided across layers
Protection from application mistakes	Lower if database constraints are limited	Stronger for rules also protected by database constraints
Complexity	Potentially simpler initially	Requires careful allocation of responsibilities
Concurrency	Application must appropriately manage conflicting updates	Application logic can be combined with database concurrency mechanisms

2.3 CivicConnect Recommendation

2.3.1 Recommended Persistence Approach

CivicConnect should use a layered persistence model with application-level business logic provided by database transactions, integrity constraints and the right concurrency control.

Updating the status of the request and creating the status change history record should be considered as one atomic operation for the selected status change operation. If both operations are successful, the transaction can be committed. Either operation will result in rolling back the transaction if it fails, so CivicConnect does not have a status change without an associated history record or a history record for a status change that never happened.

This is a better choice than application-level controls, as CivicConnect will be using the request history for traceability and accountability. (Cui, et al., 2024) have shown that transaction behaviour is important in ensuring the consistency of databases, and (Huang, et al., 2023) have indicated that database constraints can also help protect databases against integrity issues that are not preventable by application-level validation.

An optimistic concurrency-control approach is recommended for concurrent updates. A request may include a version number or other means to determine if the request has changed since it was retrieved by a staff member. CivicConnect can compare the anticipated version to the existing version that it stores prior to accepting an update. If a different staff member has made a change to the request the old change should not be accepted over the newer one; nor should it be silently overwritten.

This is like CivicConnect, as staff should generally be able to work on their own without requests locked for extended periods of time and conflicting updates should be caught and safely dealt with.

2.3.2 Responsibility Allocation

Data integrity should be distributed among the system and not be the only duty of one layer.

The user interface should provide basic input validation (e.g., show missing fields, show incorrect formats for a field, etc.) and should give the user immediate feedback. But don't consider client-side validation as the last line of defence to the stored data.

Business rules in CivicConnect should be enforced in the application/service layer. This means that it is checking if the user who is authenticated has the permissions to perform the requested

action and whether the proposed status transition is allowed, whether the information needed to complete the action has been provided. It should also ensure the persistence operation and act accordingly on detecting a concurrency conflict.

The database layer should protect fundamental persistent-data integrity. Structurally invalid data should be prevented by appropriate constraints, and the changes of related items like history insertion and the update of status should be committed or rolled back together in a transaction. Concurrently accessed databases should allow the application to detect or prevent conflicting updates.

This allocation allows CivicConnect's business rules to remain in the application and provides the database as an extra level of integrity. It thus steers clear of both extremes of relying on the database to protect all the persistent data or relying entirely on the application to protect all the persistent data.

2.3.3 Milestone 2 Decision and Artefact Linkage

The Assignment 2 research is evidence that supports a persistence decision to be formally assessed and implemented during Milestone 2. The recommended options are to implement transactional persistence for logically related changes, layered validation and integrity controls and optimistic concurrency protection for potentially conflicting request updates for CivicConnect.

This recommendation should feed into several Milestone 2 engineering artefacts. The chosen persistence strategy may be documented in an Architecture Decision Record (ADR) or Engineering Decision Log along with alternatives that were considered and why the selected strategy was chosen. The data design should illustrate the relationship between the current record and the status/history data; the implementation should illustrate the transaction boundary and the selected concurrency mechanism.

The decision should also yield measurable results. For instance, Milestone 2 testing should cover that an insertion to the history record fails does not leave an uncompleted history record partially modified, and that a stale concurrent modification cannot quietly replace a fresh one. These tests would give evidence that the persistence decision investigated in Assignment 2 is accurate.

This establishes a clear path for research to implementation:

A2 research → compare persistence alternatives → recommend approach → M2 records decision
→ implementation applies it → tests provide evidence

3. APIs AND INTEGRATION

3.1 Integration Problem

3.1.1 Selected CivicConnect Interaction

The integration selected for CivicConnect is the core CivicConnect platform and a mock WhatsApp notification service.

The goal of this integration is that once the CivicConnect users are notified of the need for a request, it can be communicated to them via a “WhatsApp-like” channel without the project team having to build the full production “WhatsApp Business” platform at this point in project.

A good example of such a request is to have a status changed. If an authorised member of staff updates a service request to Resolved, CivicConnect may show a message to the person who has requested the service indicating that the request has been completed. The notification might include the request reference number, new status and a brief status message.

The reason this interaction was chosen is that it is a true problem of integration in an external service. CivicConnect must decide how it is going to talk to the simulated notification service, what to do in the event of failure, and the degree of dependency between the two.

The question, then, is not just about whether CivicConnect should notify, but what they should notify. The engineering issue is the design of communication from CivicConnect to the communication component on the outside.

3.1.2 Producer / Caller and Consumer / Callee

In this interaction, the CivicConnect application is the producer/caller, as it creates the notification requirement when a triggering event occurs, for example, a request status change.

The simulated WhatsApp notification service is a consumer or callee of the notification service – it receives information about the notification and is responsible for simulating the delivery of the notification to the requester.

For example:

Simulated WhatsApp service sent by CivicConnect notification request.

The CivicConnect component is aware that a notification should be sent, and the notification component is the one doing processing or delivery of the notification message.

Depending on the subsequent integration strategy, the relationship between the two parts will vary. In the synchronous approach, CivicConnect could directly invoke the notification service, and wait for an immediate reply. An asynchronous approach might involve CivicConnect publishing or queuing the notification request and having the notification service process the request separately.

3.1.3 Information Exchanged

Only the data needed in the context of the notification operation should be shared. The message can contain:

Information	Purpose
Request identifier	Identifies the relevant CivicConnect request
Requester contact identifier	Identifies the intended notification recipient
Notification type	Indicates the purpose of the message, such as status update
New request status	Communicates the updated state, for example Resolved
Message content	Provides the user-facing notification text
Event/reference identifier	

	Helps track the notification and prevent confusion between messages
Timestamp	Records when the notification event was generated

This integration shouldn't be sending any unnecessary CivicConnect data. The only information that crosses the integration boundary should be enough for the notification.

This is especially crucial because CivicConnect might include service-request information which is not necessary for the notification component. Restriction of exchanged data minimises the need for the systems to be dependent on each other and the information being shared outside the main application.

The message format and API contract should only be determined after the integration mechanism is decided.

3.1.4 Failure Consequences

Failure to integrate may happen even if the core CivicConnect business application has been successful.

For instance, a staff member could update Request #105 to Resolved, but the simulated WhatsApp service may not be available at the time CivicConnect tries to send the notification.

Failure of the external notification service may cause the CivicConnect operation to be delayed or otherwise disrupt the integration. A requester might also not get any notification even though the request is successfully updated.

Other possible failures include delayed delivery, duplicate notifications, incorrect message content, network communication failure or an invalid response from the external service.

The consequences are different depending on the type of failure. If the notification fails it does not necessarily imply that the service request failed to travel the lifecycle. But the failure to notify should also be easily visible and controllable since otherwise the user wouldn't know that his request has changed.

This helps to distinguish between two scenarios:

Success: The status for the request was successfully changed.

The notification was successfully sent or handled.

This allows for treating those as separate results, so that an external communication failure doesn't be equated with a failure in CivicConnect's core request-management operation.

3.1.5 Need for an Explicit Integration Decision

There is an explicit integration decision that must be made since the engineering consequences are different for different communication mechanisms.

A synchronous request/response model might be easier to grasp and an immediate way for CivicConnect to determine if the notification request was accepted or not. The CivicConnect operation, however, can be more reliant on the external service availability and response time.

That immediate dependency can be minimized with an asynchronous messaging, as CivicConnect can record the notification event, or publish the event, and let another component process it at its own pace. Studies on asynchronous communication have demonstrated that this approach can enhance resilience and decrease dependency on runtime between communicating components but can also add complexity in other aspects like message handling, tracking delivery and recovery from failures.

The challenge for the integration decision is thus to take more than technical preference into account. It should assess coupling, performance, failure mechanisms, testability, deployment complexity, boundaries of security and compatibility with future changes.

3.2 Integration Alternatives

3.2.1 Integration Approach 1

The first one is synchronous communication via REST API. This would involve CivicConnect contacting an endpoint offered by the simulated WhatsApp notification service each time a notification is required.

For instance, once Request 105 is set to Resolved, CivicConnect could take an HTTP request with the proper notification data to the notification service. The request would be processed and either a response would be returned showing that the request was accepted or an error message would be returned.

The way of doing it is simple request-response interaction. It is easy to understand why REST APIs have been so popular for communicating between client and services as they are simple, scalable and work across platforms (Tanveer, et al., 2025).

One disadvantage of this is that the two components become temporally dependent when they interact. At that time, CivicConnect must talk with the notification service and await a reply. If the service is not available, or it takes a long time to respond, CivicConnect should wait, try again, or return an error.

3.2.2 Integration Approach 2

The second option is message-asynchronous communication. CivicConnect would create a notification message or event that can be processed independently of CivicConnect waiting for the notification service to process a request.

For instance, if Request 105 is Resolved, CivicConnect would create a notification event with the identifier, who is being provided with the notification, and the pertinent details of the notification message. That event would then be consumed and processed by a simulated WhatsApp component.

The biggest benefit is that the core CivicConnect operation doesn't need to wait for the notification service to complete the notification. This alleviates the current runtime coupling of the two parts.

The primary reason why resilience is highlighted in research on asynchronous communication is that it can keep parts of the system from being affected by some of the failures in the rest of the system (Bosilia, et al., 2025). Asynchronous communication adds to the engineering challenge,

however, as well as the need for message processing, failure handling and the prevention of message duplication or loss.

3.2.3 Coupling and Performance

Altogether the two options give rise to different types of coupling.

With synchronous REST communication, CivicConnect needs to know where and how to call the notification service and requires it to respond during the interaction. This leads to more temporal coupling as both parts need to be equally available at the same time.

The synchronous approach is still efficient for simple interactions, however. CivicConnect gets instant response without the need for extra messaging infrastructure. In the case of a comparatively small application, this simplicity can be advantageous.

Asynchronous communication means that both can't be simultaneously present. CivicConnect can generate the notification event and go ahead and run while the notification component is processing it. This could affect the responsiveness of the core operation, especially if the external service is slow.

But this advantage is gained only with extra infrastructure and processing. There must be a queue, event mechanism or other intermediary that must be controlled, and notice delivery can't happen in real time.

So, the question of performance is not just about which is faster. CivicConnect needs to decide which is more valuable, immediate or reduced reliance on notification-processing time, for the selected interaction.

3.2.4 Failure Behaviour and Reliability

One of the major differences between the alternatives is failure behaviour.

With synchronous communication, failure of the simulated WhatsApp service is immediately visible to CivicConnect. The application might return some error message or timeout and let the appropriate component know that sending the notification failed.

The downside is that the external failure does impact the caller. CivicConnect assumes that the overall operation is complete when it receives the notification service, so if the notification service is slow to respond, or it fails to respond, it could affect the user's experience although the request status was successfully updated.

These failures can be split by asynchronous communication. CivicConnect was able to successfully complete the status change and log a notification event even if the notification processor was temporarily unavailable. The notification could then be executed when the component appears again.

This adds to reliability responsibilities, though. These must be considered by the system such as failed message processing, retries, duplicate processing and messages that are repeatedly failed.

As such, failure cannot be cured with asynchronous communication. Rather, it alters the location and way failure needs to be dealt with.

3.2.5 Testability and Deployment

Relative ease allows a synchronous REST integration to be tested. The development team can simulate the notification endpoint and ensure CivicConnect makes the right request and returns the proper response in both success and failure cases, as well as when a time limit is exceeded.

The deployment is also relatively easy, as it does not require many additional components. This might be useful for CivicConnect because the project is in the developmental phase by a small group of students and if there was unnecessary infrastructure it might increase the amount of effort to implement.

The integration is done asynchronously and needs further testing. The team would have to confirm that events are generated properly, consumed successfully, and dealt with properly if there is a failure in processing. Duplicate messages, delayed processing and retry behaviour may also need to be considered when testing.

The same applies for deployment, as it may also need to involve another queue, broker or event processing system.

In this regard, the asynchronous communication solution could offer architectural advantages, but this must be considered against the increased implementation, testing and deployment complexity that is added to CivicConnect.

3.2.6 Security Boundaries

Regardless of the integration mechanism used, since the information is passed from the CivicConnect core application to another component, a security boundary is created with the notification component.

In the case of a REST integration the API should validate and authorize inputs and ensure that the request is properly authenticated and authorized. Among important categories of REST API security vulnerabilities, (Tanveer, et al., 2025) list the authentication, authorisation and input-validation weaknesses. The vulnerabilities in REST API can have impacts on the confidentiality, integrity and availability of Web services.

So, it is important that CivicConnect does not release any unnecessary service request information via the integration. To cross the boundary, only information needed for production of the notification must cross the boundary.

Similar requirements are generated during asynchronous communication. The rights to access the messaging mechanism shall be controlled, to prevent unauthorised components from publishing, consuming or modifying notification messages.

Under either one of these approaches' security is required. The mechanism doesn't eliminate the need for security controls but changes their positioning and kind.

3.2.7 Versioning and Change Compatibility

The integration should also be manageable as either part changes, and this is why it's important to keep it within a scope that one can manage.

Adopting the REST approach assumes an agreed API contract between CivicConnect and the notification service. Changes to endpoints, request structures or expected responses could

break the integration if compatibility is not maintained. Poor management of API assets, such as unsupported or unsafe API versions, is also an API-security concern, according to Tanveer et al. (2025).

If the REST interface should change, then it would require a clear interface contract, along with the implementation of controlled versioning.

An agreed upon message contract is also crucial for asynchronous communication. The structure or meaning of notification events may change and the consumer may not be able to process the change correctly. Producers and consumers should hence refrain from incompatible changes in the message format unless they coordinate how the current versions will be processed.

Both options, therefore, need to be compatible managed. API contracts and versions need to be managed for REST, event or message schemas for asynchronous communication.

3.2.8 Comparison of Alternatives

Consideration	Synchronous REST API	Asynchronous Messaging
Communication	Direct request-response	Message/event processed separately
Temporal coupling	Higher	Lower
Response	Immediate success/error response	Processing result may be delayed
External service outage	Immediately affects API call	Core operation can potentially continue
Reliability handling	Timeouts, errors and retries	Retries, duplicate handling and failed messages
Implementation complexity	Lower	Higher
Infrastructure	Relatively simple	Queue/event mechanism may be required
Testing	Easier direct interaction testing	More scenarios and components to test
Security	Secure API endpoint and exchanged data	Secure producer, consumer and messaging mechanism
Change compatibility	API contract/version management	Message/event schema management
Suitability for small project	Easier to implement and demonstrate	More robust separation, but greater complexity

Both are not necessarily good. The simplicity, prompt feedback and reduced infrastructure of synchronous REST communication are useful for a smaller project. Asynchronous messaging offers a looser coupling at runtime and a more robust failure isolation while adding complexity to implementation and operation.

The trade-off between CivicConnect's simplicity of direct REST communication and its failure isolation and reduced temporal coupling of asynchronous communication is therefore key.

3.3 CivicConnect Recommendation

3.3.1 Recommended Integration Approach

The suggested solution for CivicConnect is a synchronous REST API integration with a simulated WhatsApp notification service, for the duration of this project.

This approach is recommended to show CivicConnect integration with external services without adding the complexity of an asynchronous messaging solution. Once the relevant event has taken place, e.g., the request has been changed to Resolved, CivicConnect can send the required information of the notification to the simulated WhatsApp service via a pre-defined API endpoint and receive a response immediately as to whether the request has been accepted.

Do not require that the service-request update be successful to be fulfilled. The result of the notification should not be part of the core CivicConnect operation. If the notification service is not available, the request status should be correctly updated, and the notification failure should be noted and dealt with separately.

This gives a realistic compromise between architectural simplicity and complexity.

3.3.2 Rationale and Trade-offs

Proportionality is the primary reason for choosing synchronous REST communication. CivicConnect is a relatively small-scale academic project, the team needs to prove effective engineering choices without adding unnecessary technical complexity.

This synchronous REST approach has several advantages for the current project. It is easy to implement, test and demonstrate, gives an instant response to the simulated notification service and it does not require configuring and maintaining a separate message broker or queue.

This simplicity has its drawbacks however, as demonstrated in Section 3.2. The synchronous communication is more time coupled since CivicConnect communicates directly with the notification service and awaits a response. The API call might fail or time out if the external service is slow or unavailable.

Therefore, CivicConnect must be flexible in its design to ensure that if the notification fails, the service request operation will not be invalid. The impact of failures in external services can be minimized by using appropriate timeout handling, error reporting and controlled retry logic.

The asynchronous option provides more separation of the core system and notification processing and can result in greater resilience when notification volume or failure of external services becomes more prominent. But would add further implementation, testing and deployment complexity that is not warranted by the size of the project.

The chosen REST approach is thus a conscious compromise:

CivicConnect will tolerate a slightly higher runtime coupling that will require less complexity in implementation, easier testing, and better visibility into the integration requirement.

Not necessarily the end of asynchronous messaging. CivicConnect can still be used as a future architectural solution when it will need more notification volumes, more failure isolation and/or more independent processing.

3.3.3 Milestone 2 Architecture / ADR Linkage

The research in the Assignment 2 should directly influence the integration architecture in Milestone 2.

A recommended integration decision should be documented in a form of a formal Architecture Decision Record (ADR) or Engineering Decision Log entry. The record should include identification of the integration problem, the alternatives that were explored, the chosen synchronous REST approach, and explanation of the reasoning behind this choice and the significant trade-offs that were made.

The Milestone 2 architecture should now display CivicConnect interacting with a mocked-up WhatsApp notification service via a defined API interface. The interface should indicate the information that needs to be notified, what it is expected to do, and what will happen if the notification service is unreachable.

Implementation and testing should give evidence that there is a correct application of the decision. For instance, Milestone 2 should show that CivicConnect can issue a valid notification request, process it when it is accepted, safely process it if it times out or fails, and maintain that if the notification service fails, then the original request that led to the notification is not reversed or corrupted.

4. Collaborative Engineering and Continuous Integration

4.1 Software Configuration Management and Collaborative Integration

4.1.1 Version Control vs Software Configuration Management

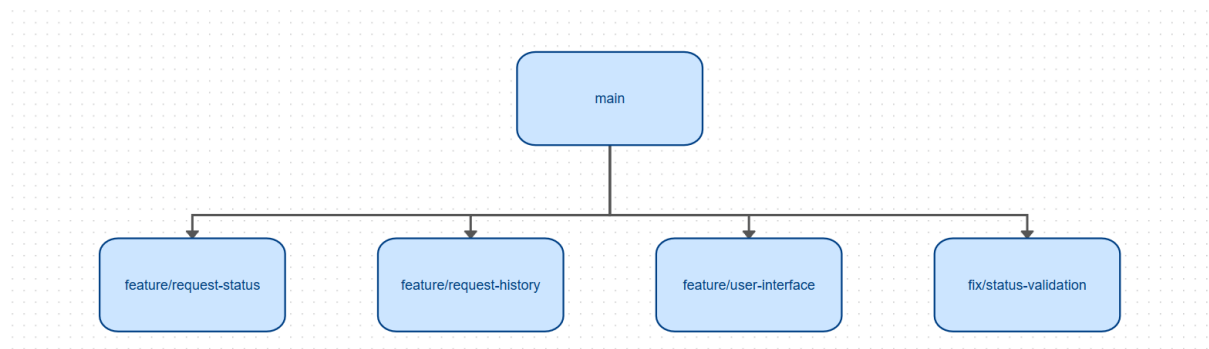
Version control and Software Configuration Management (SCM) are closely related concepts, but they do not represent the same level of engineering control. Version control primarily manages changes made to files and source code over time. It allows developers to retain a history of changes, work on different versions of a system, compare modifications and restore previous states when required.

Software Configuration Management has a wider scope. The Software Engineering Body of Knowledge describes SCM as involving the identification, control, tracking and auditing of changes to software artefacts across the software lifecycle. SCM therefore includes version control but also concerns the integrity, traceability and controlled modification of configuration items such as source code, requirements, documentation, dependencies, builds and releases.

For CivicConnect, Git and GitHub may provide version-control functionality, but simply storing the project in a Git repository would not by itself provide complete configuration management. The team must also determine how changes are proposed, reviewed, approved, integrated and traced back to requirements or engineering decisions. This distinction is important because several team members may modify related parts of CivicConnect simultaneously, and uncontrolled changes could result in defects, inconsistent baselines or loss of traceability.

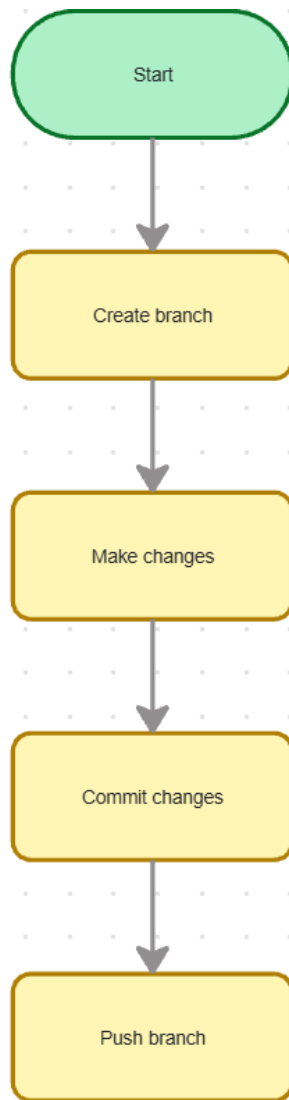
4.1.2 Branching Strategy

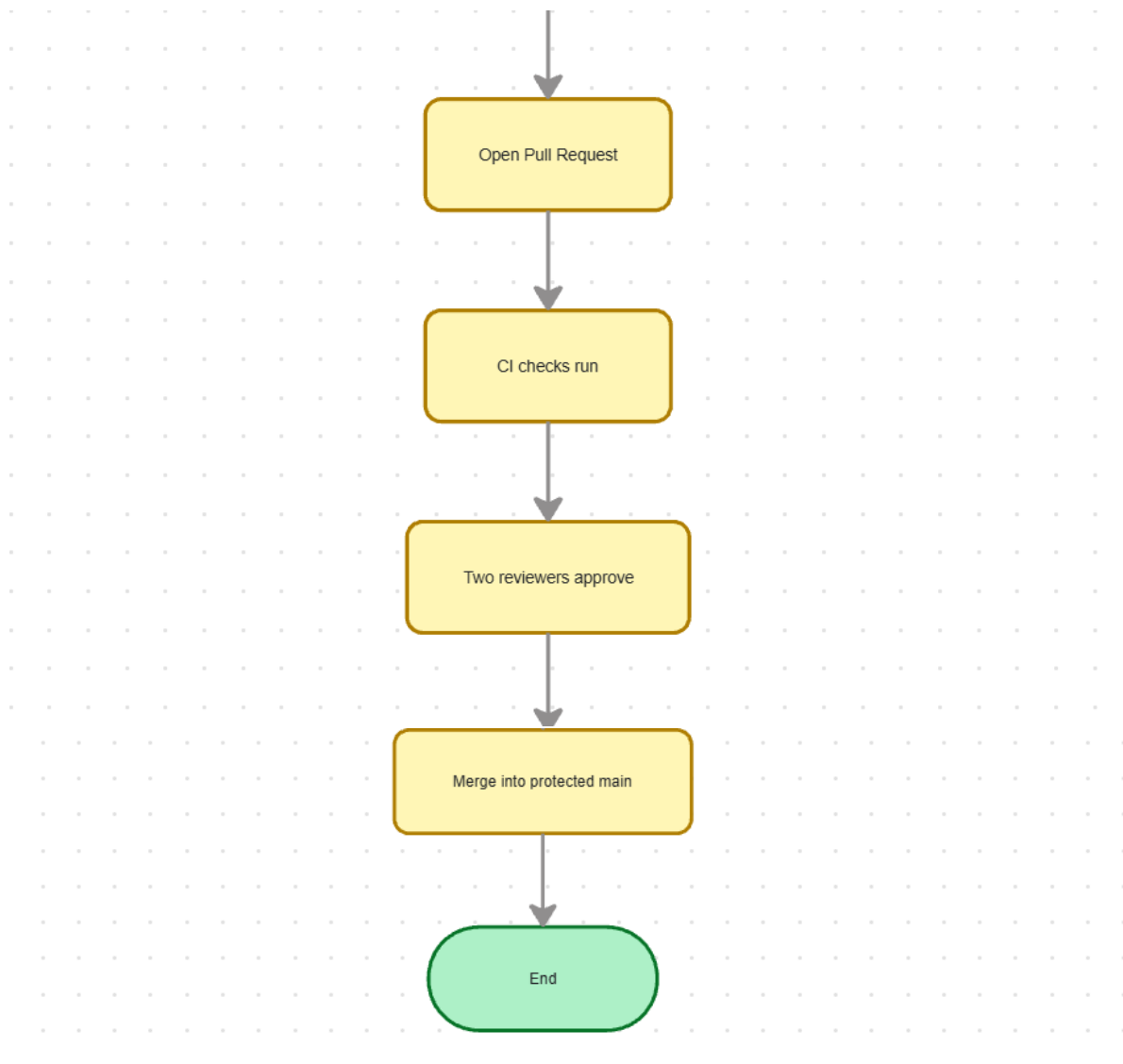
A simple feature-branch workflow makes more sense:



Nobody develops directly on `main`.

Instead:





A lightweight feature-branch strategy is appropriate for CivicConnect because the development team consists of only three members. Each meaningful change should be developed on a separate short-lived branch created from the current `main` branch. Suitable branch names could include `feature/request-status`, `feature/request-history` or `fix/status-validation`.

This approach isolates unfinished work from the controlled project baseline and allows changes to be reviewed before they become part of the main system. Long-lived and highly complex branching models would introduce additional merge and management overhead that may not provide sufficient benefit for a three-person student team.

The `main` branch should therefore represent the currently approved and integrated CivicConnect baseline rather than an individual developer's working area.

4.1.3 Pull Requests and Peer Review

Changes to CivicConnect should be integrated into `main` through Pull Requests rather than direct pushes. A Pull Request provides a controlled review point where the proposed change, associated commits and automated verification results can be examined before integration.

The CivicConnect project requires two reviewers before changes may be merged into the protected `main` branch. GitHub branch-protection rules support this type of workflow by allowing repositories to require a Pull Request and a specified number of approving reviews before merging. Required status checks may also be configured so that changes cannot be merged while mandatory automated checks are failing.

For the three-person CivicConnect team, this means that the developer who creates the change should not approve their own work. The other two team members should review the Pull Request. This ensures that all three members remain familiar with the evolving system while also reducing the likelihood that unreviewed defects or architectural inconsistencies enter the controlled baseline.

4.1.4 Traceability

Traceability is important within collaborative development because a software change should not exist without a clear reason for why it was introduced. For CivicConnect, commits and Pull Requests should therefore be linked where possible to the requirement, issue, risk, Architecture Decision Record (ADR), or other engineering decision that motivated the change.

GitHub supports linking Pull Requests to issues, including through keywords such as `Closes`, `Fixes` or `Resolves`. This provides a visible connection between the work being requested and the code change used to implement it (GitHub, n.d.-a).

For CivicConnect, Pull Request descriptions should therefore identify the relevant requirement or engineering artefact where applicable. For example, a Pull Request implementing controlled status transitions could reference the appropriate functional requirement and any persistence ADR associated with that behaviour. Meaningful commit messages should also describe the purpose of the change rather than only stating that files were modified.

This creates a traceable path such as:

Requirement / Issue → Branch → Commits → Pull Request → Review → CI Evidence → Merge

Such traceability supports Software Configuration Management because the team can determine not only **what changed**, but also **why it changed**, **who reviewed it** and **what verification occurred before it entered the controlled baseline**.

4.1.5 CivicConnect Collaboration Risk

A significant collaboration risk for CivicConnect is that one team member may modify request lifecycle behaviour, persistence logic or interfaces while another team member continues developing against an earlier assumption.

For example, one developer may change how a request moves from *Assigned* to *Resolved*, while another developer is simultaneously implementing the user interface or reporting logic based on the previous behaviour. If these changes are integrated without appropriate coordination and verification, the system may compile successfully while still containing inconsistent behaviour between components.

This risk is particularly relevant because CivicConnect's persistence research has already established that request status and request-history information must remain consistent. The current Task 2 recommendation therefore treats related status/history changes as a controlled operation rather than independent updates.

The collaboration risk is therefore not limited to textual Git merge conflicts. A **semantic integration conflict** may occur even when Git is technically able to merge the files. The resulting program could contain incompatible assumptions, incorrect behaviour or regression defects.

4.1.6 Comparison of Risk Controls

Engineering consideration	Ad-hoc/direct integration	Controlled PR + review workflow
Development speed	Initially faster because developers can push changes immediately	Slightly slower due to review and CI
Main branch protection	Low	High
Merge conflict detection	Often occurs during or after integration	Changes are reviewed before integration
Semantic conflict detection	Depends heavily on developer awareness	Reviewers can identify incompatible assumptions
Traceability	Limited	PR, commits, issues and reviews create evidence
Knowledge sharing	Lower	Review exposes changes to other team members
Automated verification	Optional/manual	Can be mandatory before merge
Administrative overhead	Low	Moderate
Suitability for CivicConnect	Weak once multiple members integrate related work	Stronger and proportionate for a three-person team

The direct-integration approach has the advantage of low process overhead, which may appear attractive for a small student project. However, it provides limited protection against

unreviewed or incompatible changes and creates weaker evidence of how a change entered the baseline.

A Pull Request workflow introduces additional effort because developers must prepare changes for review and address feedback before merging. However, GitHub Pull Request reviews allow reviewers to comment, suggest modifications, approve changes or request further changes. Reviews therefore provide both defect detection and knowledge sharing within the team (GitHub, n.d.-b).

For CivicConnect, the controlled approach is therefore more appropriate because the project already requires a protected `main` branch and two reviewers. The additional process cost is proportionate to the benefit of preserving the integrity and traceability of the project baseline.

4.2.1 Purpose of Continuous Integration

Continuous Integration (CI) should not be confused with either version control or automated deployment. Git provides version-control functionality by recording changes to the project, whereas CI concerns the frequent integration and automated verification of those changes.

Fowler (2024) describes Continuous Integration as a practice in which team members integrate changes frequently and each integration is verified by an automated build and tests. GitHub similarly explains that CI workflows can automatically build and test changes so that errors are identified earlier in development.

For CivicConnect, CI should therefore provide an automated verification stage between a developer proposing a change and that change becoming part of the approved `main` branch.

CI is also different from Continuous Deployment. CivicConnect does not need to automatically deploy every successful change in order to practise Continuous Integration. A CI pipeline that builds, tests and verifies Pull Requests is sufficient for the research objective.

4.2.2 CI Triggers

The CivicConnect CI workflow should primarily be triggered when:

1. a Pull Request targeting `main` is created or updated; and
2. a change is successfully merged into `main`.

Running CI on Pull Requests provides feedback before the proposed change enters the controlled baseline. Running it again after integration provides confirmation that the resulting `main` branch remains valid.

GitHub Actions supports workflows triggered by repository events such as pushes and other development events, and its CI results can be displayed directly within Pull Requests (GitHub, n.d.-c).

A scheduled pipeline is not necessary for the basic CivicConnect workflow because the main risk being controlled is the integration of new changes rather than a time-based operation.

4.2.3 Repeatable Builds

A CI build must be repeatable so that the result does not depend on a particular developer's computer.

For CivicConnect, the repository should therefore contain everything necessary to recreate the build except confidential information. This should include:

- source code;
- dependency manifests and appropriate lock files;
- build instructions/scripts;
- test code;
- required configuration templates; and
- documented runtime/tool versions where necessary.

The CI environment should install dependencies from the project's declared configuration and then execute the same defined build and test process each time.

This reduces the risk of the classic "**it works on my machine**" situation, because a successful build must also work within a clean automated environment.

Fowler (2024) identifies automated builds and self-testing builds as central CI practices.

4.2.4 Automated Checks

CivicConnect's CI pipeline should perform several automated checks appropriate to the implementation stack eventually selected.

A minimum pipeline should include:

Automated check	Purpose
Build / compilation	Detect syntax, dependency and compilation failures
Unit tests	Verify important application behaviour
Linting / static analysis	Identify code-quality and common programming problems
Dependency/security checking	Identify known vulnerable dependencies where appropriate
Integration tests where practical	Verify interactions between important components

GitHub identifies builds, tests, linters, security checks, code-coverage checks and functional tests as examples of checks that may form part of CI.

Not every possible automated check should be added merely because it exists. The selected checks should relate to CivicConnect's actual technologies, risks and requirements.

4.2.5 Blocking Checks vs Warning Checks

Not every CI result needs to have the same consequence.

Blocking quality gates

The following should normally prevent a Pull Request from being merged:

- the application does not build or compile;
- required unit tests fail;
- critical automated tests fail;
- a serious security/dependency problem is detected;
- required CI configuration itself fails.

These checks provide evidence that the proposed change may make the controlled baseline invalid or unsafe.

GitHub branch protection allows required status checks to prevent merging until the required checks have passed.

Warning checks

Some checks may initially provide warnings rather than block integration, such as:

- minor formatting/style issues;
- an initial code-coverage target while the test suite is still growing;
- lower-severity maintainability findings;
- experimental analysis tools not yet stable enough to act as quality gates.

The distinction prevents the team from creating a pipeline so strict that minor issues unnecessarily stop all work, while still protecting main from changes with significant correctness or security problems.

As the CivicConnect implementation matures, warning checks can be promoted to blocking gates when the team has sufficient evidence that they are reliable and valuable.

4.2.6 Secrets and Configuration

Sensitive configuration must not be stored directly inside source files or committed to the CivicConnect repository.

Examples could include future API credentials, database passwords, tokens or deployment credentials.

GitHub Actions supports encrypted repository, organisation and environment secrets. These secrets are made available to a workflow only when they are explicitly referenced by that workflow (GitHub, n.d.-d).

CivicConnect should therefore separate:

Normal configuration → documented configuration files/templates

from

Sensitive configuration → secure secret storage supplied at runtime.

This reduces the risk of exposing credentials through Git history and also makes configuration differences between development and later environments easier to control.

4.2.7 Visibility of CI Results

Automated verification is useful only if developers and reviewers can see its outcome.

CI results should therefore be visible directly on the CivicConnect Pull Request. A reviewer should be able to determine whether the build succeeded, whether tests passed and whether any required checks remain incomplete before approving the change.

GitHub exposes automated checks and workflow results within the Pull Request interface and supports using required status checks as merge conditions.

This provides evidence that the change was verified rather than relying on a developer stating that they tested it locally.

4.2.8 Automation vs Human Review

CI automation should support human peer review rather than replace it.

Automated checks are particularly effective at repeatedly identifying objective conditions such as compilation failures, failing tests, linting violations and selected known security problems. However, automated checks cannot independently determine whether every software-engineering decision is appropriate for CivicConnect.

Human reviewers are still required to consider matters such as:

- whether the implementation actually satisfies the intended requirement;
- whether responsibilities have been placed in the appropriate component;
- whether unnecessary coupling has been introduced;
- whether security and privacy implications have been considered;
- whether the change remains understandable and maintainable; and
- whether associated documentation or ADRs require modification.

GitHub's own Pull Request model treats review as a process in which team members examine proposed changes, provide feedback and determine whether changes are ready to merge.

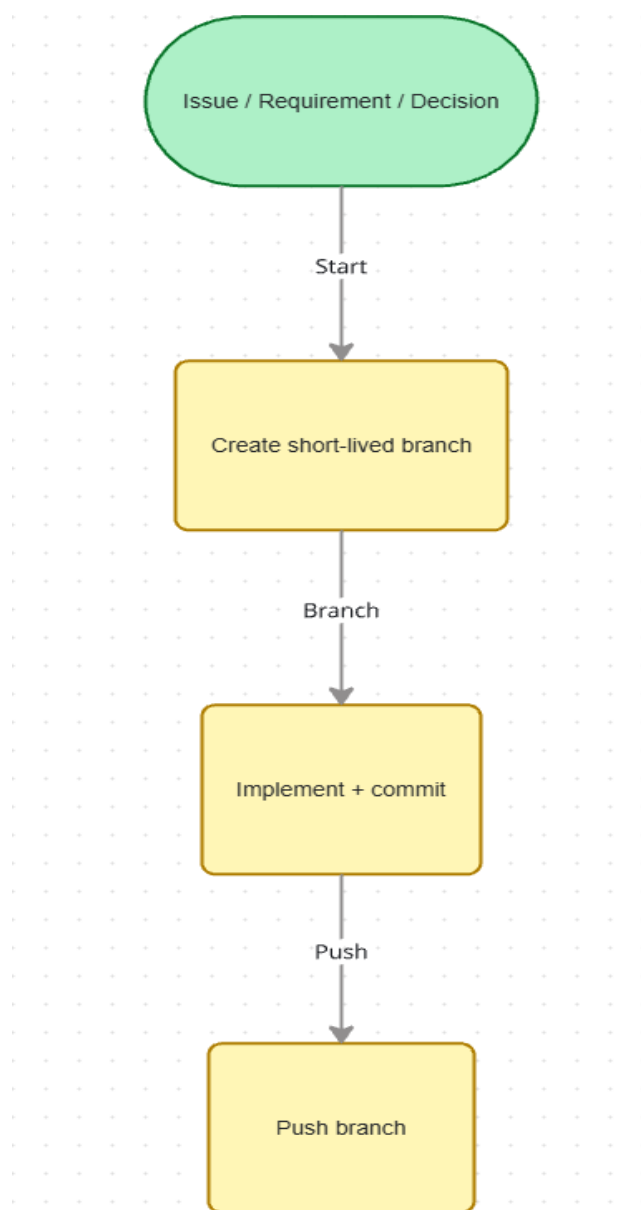
For CivicConnect, a green CI result should therefore mean **"the automated checks passed"**, not **"the change is automatically correct."**

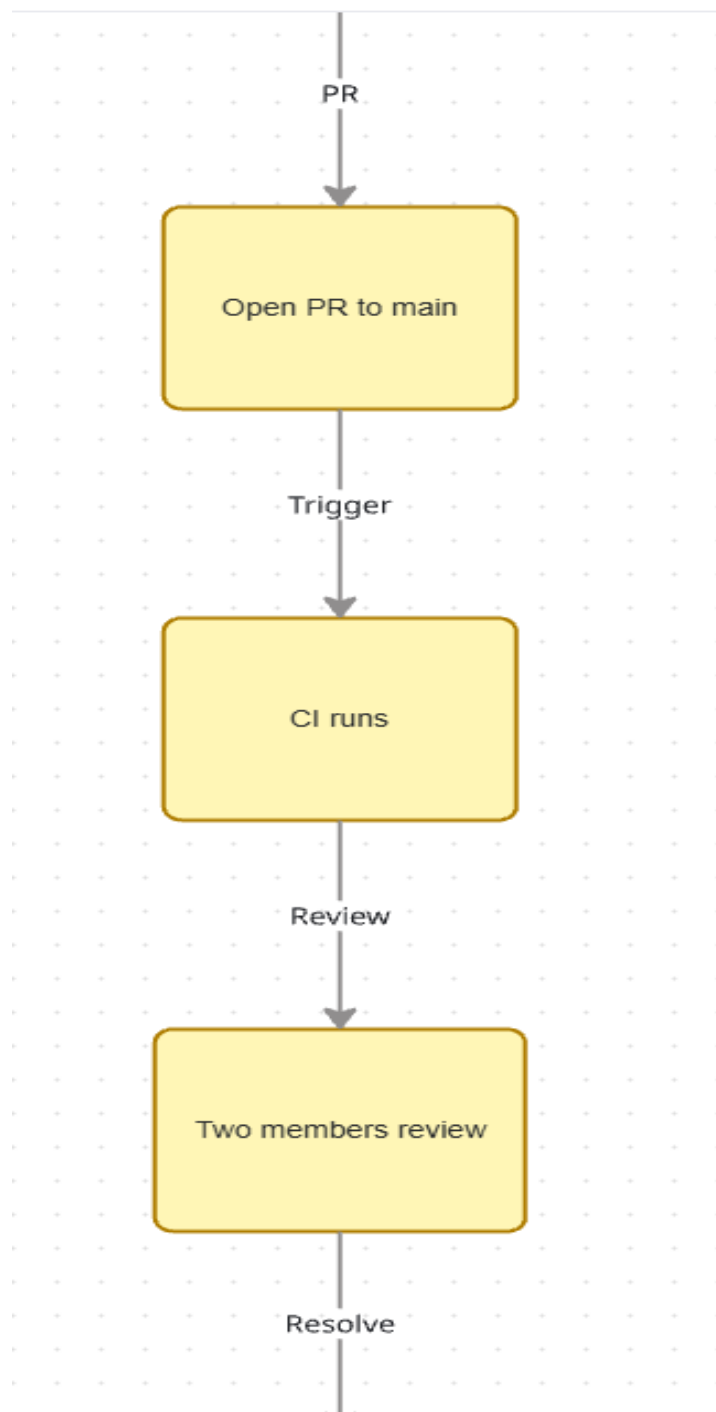
4.3 Recommended CivicConnect Team Control

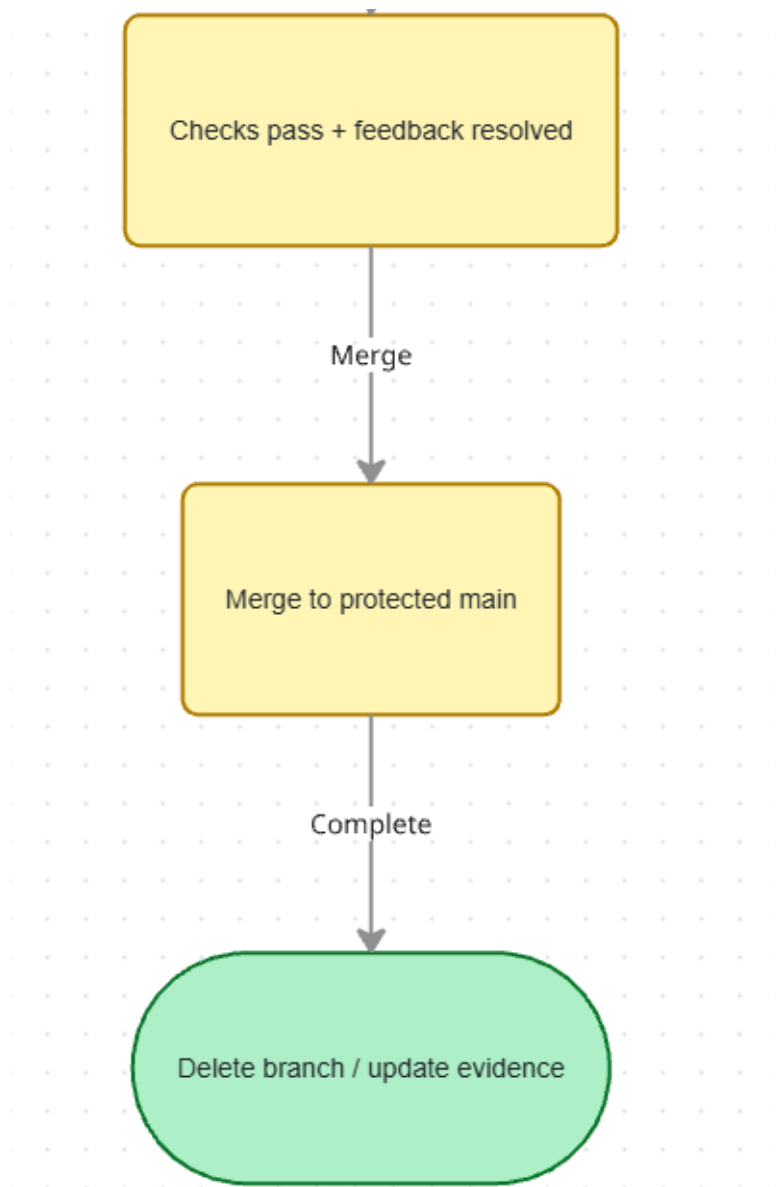
4.3.1 Recommended GitHub Workflow

CivicConnect should adopt a lightweight protected-main workflow suitable for its three-person team.

The recommended process is:







Direct development on `main` should therefore be avoided. The developer responsible for a change works on a short-lived branch and submits a Pull Request when the change is ready for integration.

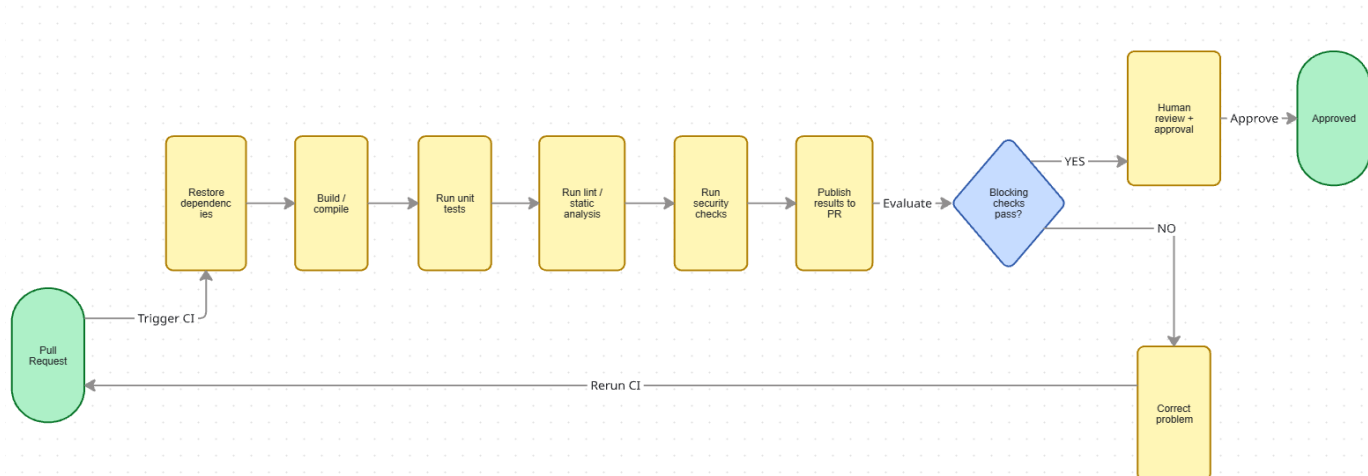
The Pull Request should contain a concise explanation of the change, its reason and links to relevant requirements, issues, ADRs or other project evidence where applicable. GitHub supports both required Pull Request reviews and required status checks on protected branches.

Because CivicConnect consists of three members and the project requires **two reviewers**, the two team members who did not author the change should review it before integration.

4.3.2 Recommended CI Workflow

The recommended CivicConnect CI process should run automatically against Pull Requests targeting main.

A proportionate initial workflow is:



The build and core test stages should be treated as mandatory quality gates. Serious security failures should also block integration. Less mature checks, such as early code-coverage thresholds or minor style findings, may initially generate warnings and can later become blocking requirements as the project matures.

Sensitive values used by automated workflows should be provided through appropriate secret/configuration mechanisms rather than committed to the repository.

4.3.3 Risk Control and Rationale

The recommended combination of protected branches, short-lived development branches, Pull Requests, two-person review, traceability and automated CI checks provides a proportionate control mechanism for CivicConnect.

It addresses the identified collaboration risk in several ways. Branch isolation prevents incomplete work from immediately affecting main; Pull Requests expose changes to the team before integration; peer review can identify incompatible assumptions or design problems; and automated CI can detect repeatable build and test failures.

The approach also avoids unnecessary complexity. CivicConnect has only three developers, so a heavyweight configuration-management process or complex long-lived branching model would introduce administrative overhead that is difficult to justify. The recommended workflow instead provides the controls most directly related to the team's risks and the Master Project Brief's protected-main and two-reviewer requirements.

The final repository configuration and CI implementation should be documented as project evidence in Milestone 2. Relevant evidence may include branch-protection settings, Pull

Requests, reviewer approvals, CI runs, test results, linked requirements/issues and any ADR or PED entry that records the team's chosen collaborative engineering approach.

5. RESEARCH-TO-DECISION MAP

A2 finding	Evidence / alternative considered	Recommendation	Project decision it should inform	Expected PED / ADR / RTM / application evidence
Design Problem 1	Abstract Factory Pattern was compared to Builder Pattern	Abstract Factory Pattern	M2 internal design/pattern decision	ADR/design diagram, implementation location and relevant RTM links
Design Problem 2	Observer Design Pattern was compared to Adapter Design Pattern	Observer Pattern or Observer-Adapter Hybrid	M2 internal design/pattern decision	ADR/design diagram, implementation location and relevant RTM links
Persistence / Data	Application-centred control was compared with layered application + database control. Transaction atomicity, database constraints and concurrency mechanisms were considered.	Use layered application/database responsibility; treat status + history update atomically; use appropriate integrity constraints and optimistic concurrency protection.	M2 persistence and data-integrity strategy	Persistence ADR/decision log; data model; transaction implementation; concurrency mechanism; tests demonstrating rollback and stale-update protection
Integration / API	Synchronous REST communication was compared with asynchronous messaging, including coupling,	Use synchronous REST communication with the simulated notification service for the project, while ensuring notification failure does not invalidate the core	M2 integration architecture and API/interface decision	Integration ADR; architecture diagram; API/interface definition; failure-handling implementation and integration tests

	failure behaviour, complexity and deployment consequences.	service-request update.		
SCM / CI	Direct/ad-hoc integration was compared with protected-main, feature branches, Pull Requests, peer review and automated CI verification.	Use short-lived branches, protected main, Pull Requests with two reviewers, traceability and CI quality gates before integration.	M2 repository governance, SCM and CI workflow	Branch-protection evidence; PRs and approvals; linked issues/requirements ; CI workflow file; build/test results; repository/process documentation

References

Adewole, A., 2018. *C# and .NET Core Test-Driven Development*. Birmingham: Packt Publishing.

Bosilia, N., Weinberger, G. & Haindl, P., 2025. Assessing the impact of asynchronous communication on resilience and robustness: a comparative study of microservice and monolithic architectures. *European Conference on Software Architecture*, pp. 171-186.

Chaudhry, N. & Yousaf, M. M., 2022. Concurrency control for real-time and mobile transactions: historical view, challenges, and evolution of practices. *Concurrency and Computation: Practice and Experience*, 34(3), p. e6549.

Cui, Z. et al., 2024. Understanding transaction bugs in database systems. *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1-13.

Ergun, B., 2023. *Loose Coupling and Tight Coupling in C#*. [Online]
Available at: <https://berkay-ergun.medium.com/loose-coupling-and-tight-coupling-53f5bc885667>
[Accessed 13 Sept 2026].

Huang, H., Shen, B., Zhong, L. & Zhou, Y., 2023. Protecting data integrity of web applications with database constraints inferred from application code. *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 2, pp. 632-645.

Tanveer, F., Iradat, F., Iqbal, W. & Ahmad, A., 2025. Towards secure APIs: A survey on RESTful API vulnerability detection. *Computers, Materials & Continua*, 84(3), pp. 4223-4257.

TutorialsPoint, 2026. *Cohesion in C#*. [Online]
Available at: <https://www.tutorialspoint.com/article/cohesion-in-chash>
[Accessed 13 Sept 2026].

Fowler, M., 2024. *Continuous Integration*. [Online] Available at: MartinFowler.com [Accessed 14 September 2026].

GitHub, n.d.-a. *Using keywords in issues and pull requests*. [Online] Available at: GitHub Docs [Accessed 14 September 2026].

GitHub, n.d.-b. *Giving reviews*. [Online] Available at: GitHub Docs [Accessed 14 September 2026].

GitHub, n.d.-c. *Continuous integration*. [Online] Available at: GitHub Docs [Accessed 14 September 2026].

GitHub, n.d.-d. *Secrets*. [Online] Available at: GitHub Docs [Accessed 14 September 2026].

GitHub, n.d.-e. *About protected branches*. [Online] Available at: GitHub Docs [Accessed 14 September 2026].

IEEE Computer Society, 2025. *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0a*. IEEE Computer Society.

7. AI RESEARCH AND VERIFICATION RECORD

<i>Team Member</i>	<i>AI/Tool Use</i>	<i>Purpose</i>	<i>Output Used?</i>	<i>How Independently Verified?</i>	<i>What was changed/rejected/accepted?</i>
Chance Grant	ChatGPT	Assisted with research planning, source selection, outlining and drafting support for Persistence & Data Integrity and APIs & Integration .	Partially	Sources were independently searched on Google Scholar and checked for relevance and credibility. Research claims were verified against the available academic publications and the Assignment 2 requirements.	Weak or irrelevant sources were rejected. Alternative engineering approaches were compared before recommendations were accepted.
Megan Wathen	ChatGPT/MiroDiagramAI	Used for spelling and grammar assistance, wording	Partially	The content, sequence and logic of each diagram were determined	Spelling and grammar improvements were accepted where they preserved the original

		improvements, and to generate flowchart/diagram visuals from a sequence of steps that I had already determined and typed out myself.		and checked manually against the Assignment 2 requirements and the team's research before the diagrams were included. Written content was reviewed to ensure that spelling/grammar corrections did not change the intended technical meaning.	meaning. AI-generated diagrams were accepted only when they accurately represented the manually defined workflow; any incorrect layout, wording or sequence was corrected or rejected.
Milene Field	ChatGPT/Gemini	Used to get assignment layout for planning purposes as well as direct research support for Task 1	Partially yes	Compared with other academic/standards sources, recognised professional resources, and official technological documentation to check for alignment	Table formats were accepted for formatting purposes, some content was altered and merged where facts aligned with academic/standards sources, recognised professional resources, and official technological documentation